Algorithmique I

Chapitre 6 · Variables structurées

Corrigé des exercices

Conventions. Ces exercices reprennent le chapitre sur les variables structurées [1]. L'en-tête des modules, les types simples, l'affectation, les conditions et `return` suivent le LDA 2026–2027. Pour ces exercices, on note les structures avec `type`, des champs typés et un bloc indenté. La notation pointée et la copie d'une structure entière sont celles du syllabus source. Chaque champ lu doit avoir été initialisé. Les dates et moments reçus sont supposés valides.

```
type Moment
    int heure, minute, seconde

type Date
    int jour, mois, annee
```

Exercice 1 : Une seconde plus tard

[1, p. 41]. On ajoute une seconde à une copie du paramètre. Une retenue ne monte d'un champ que si ce champ atteint sa base. Pour un moment de la journée, minuit suit la dernière seconde.

```
Moment secondeSuivante(Moment moment)
    Moment resultat
    resultat = moment
    resultat.seconde = resultat.seconde + 1
    if resultat.seconde == 60
        resultat.seconde = 0
        resultat.minute = resultat.minute + 1
        if resultat.minute == 60
            resultat.minute = 0
            resultat.heure = (resultat.heure + 1) MOD 24
    return resultat
```

Par exemple, 23:59:59 devient 00:00:00.

Exercice 2 : Comparaison de dates

[1, p. 41]. On compare de l'unité la plus importante à la plus fine : année, mois, puis jour. La première différence suffit à trancher.

```
int comparerDates(Date a, b)
    int ordre
    ordre = 0
    if a.annee < b.annee
        ordre = -1
    else if a.annee > b.annee
        ordre = 1
    else if a.mois < b.mois
        ordre = -1
```

```
    else if a.mois > b.mois
        ordre = 1
    else if a.jour < b.jour
        ordre = -1
    else if a.jour > b.jour
        ordre = 1
    return ordre
```

On obtient -1 si la première date est antérieure, 1 si elle est postérieure et 0 si les trois champs sont égaux.

Exercice 3 : Reconversions

[1, p. 41]. Une durée et un moment utilisent les mêmes champs, mais une durée peut dépasser 23 heures. On convertit d'abord toutes les unités en secondes ; dans l'autre sens, chaque reste fournit les données encore à répartir.

```
// a) Duree vers secondes
int versSecondes(Moment duree)
    return 3600*duree.heure + 60*duree.minute + duree.seconde

// b) Secondes vers duree
Moment versHMS(int total)
    Moment duree
    int reste
    duree.heure = total DIV 3600
    reste = total MOD 3600
    duree.minute = reste DIV 60
    duree.seconde = reste MOD 60
    return duree
```

12:03:40 donne 43 420 secondes, et la conversion inverse restitue la durée. On ne réduit pas les heures modulo 24.

Exercice 4 : Combien de temps ?

[1, p. 41]. On soustrait les deux moments exprimés en secondes. Si la fin précède le début sur l'horloge, on prend sa première occurrence le lendemain. Avec des moments égaux, on prend une durée nulle. Si l'on impose une fin le même jour, le cas négatif doit être exclu.

```
Moment tempsEcoule(Moment debut, fin)
    int ecart
    ecart = versSecondes(fin) - versSecondes(debut)
    if ecart < 0
        ecart = ecart + 86400
    return versHMS(ecart)
```

Entre 12:59:50 et 14:00:10, l'écart est de 3 620 secondes, soit 1:00:20.

Exercice 5 : Les points

[1, p. 42]. Un point contient ses deux coordonnées réelles. Le milieu s'obtient en moyennant chaque coordonnée ; la distance vient du théorème de Pythagore.

```
type Point
    float x, y

// a) Milieu
Point milieu(Point a, b)
    Point m
    m.x = (a.x + b.x) / 2
    m.y = (a.y + b.y) / 2
    return m

// b) Distance
float distance(Point a, b)
    float dx, dy
    dx = b.x - a.x
    dy = b.y - a.y
    return sqrt(dx*dx + dy*dy)
```

Pour $(0, 0)$ et $(3, 4)$, le milieu est $(1{,}5\,;\,2)$ et la distance est 5.

Exercice 6 : Les cercles

[1, p. 42]. Le schéma ombre une surface : on cherche donc un point commun aux deux *disques fermés*. La tangence compte, et un disque inclus dans l'autre compte aussi. Les rayons sont supposés positifs ou nuls.

```
type Cercle
    Point centre
    float rayon

Boolean disquesSeCoupent(Cercle a, b)
    float d
    d = distance(a.centre, b.centre)
    return d <= a.rayon + b.rayon
```

Si le mot *cercle* désigne seulement le contour, il faut également $d \geq |r_a - r_b|$. Cette borne exclut le cas où un cercle est strictement à l'intérieur de l'autre sans le toucher.

Références

[1] ESI, HE2B, *Algorithmique : syllabus 2025-2026*, Bruxelles, Belgique, 2025, pp. 41-42.